Microsoft

LIVE

# Python + AI

Microsoft

LIVE

**Python + AI**

🧠 Oct 7: LLMs

↖ Oct 8: Vector embeddings

🔍 Oct 9: RAG

🖼️ Oct 14: Vision models

🔢 Oct 15: Structured outputs

🛡️ Oct 16: AI Quality & Safety

🛠️ Oct 21: Tool calling

🤖 Oct 22: Agents

🔗 Oct 23: Model Context Protocol

Register @ aka.ms/PythonAI/series

Microsoft

## Python + AI

# 🔍 Retrieval Augmented Generation

https://aka.ms/pythonai/slides/rag

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

# Today we'll cover...

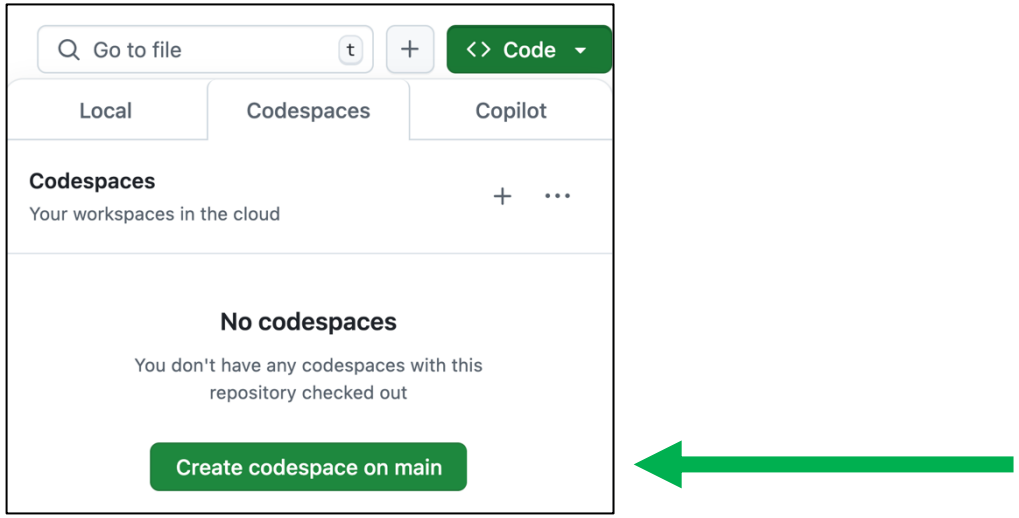
- Retrieval Augmented Generation
- RAG flows, simple and advanced
- RAG on PostgreSQL database
- RAG on documents with Azure AI Search
- More ways to build RAG

# Want to follow along?

1. Open this GitHub repository:

[https://aka.ms/python-openai-demos](https://aka.ms/python-openai-demos)

2. Use "Code" button to create a GitHub Codespace:




3. Wait a few minutes for Codespace to start up ⏱️

# Why RAG?

# The limitations of LLMs

Outdated public knowledge

**You**

Write a model class using latest version of Flask SQL Alchemy

**Assistant**

Sure! Here's an example of a model class using the latest version of Flask SQLAlchemy:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True)
    email = db.Column(db.String(120), unique=True)
```

No internal knowledge

**You**

Do my company perks cover underwater activities?

**Assistant**

To determine whether your company perks cover underwater activities, you should consult your employee benefits package or reach out to your human resources department. They will be able to provide you with detailed information regarding the specific perks and benefits offered by your company, including any coverage for underwater activities.

# Integrating domain knowledge

**Fine tuning**

Learn new skills (permanently)

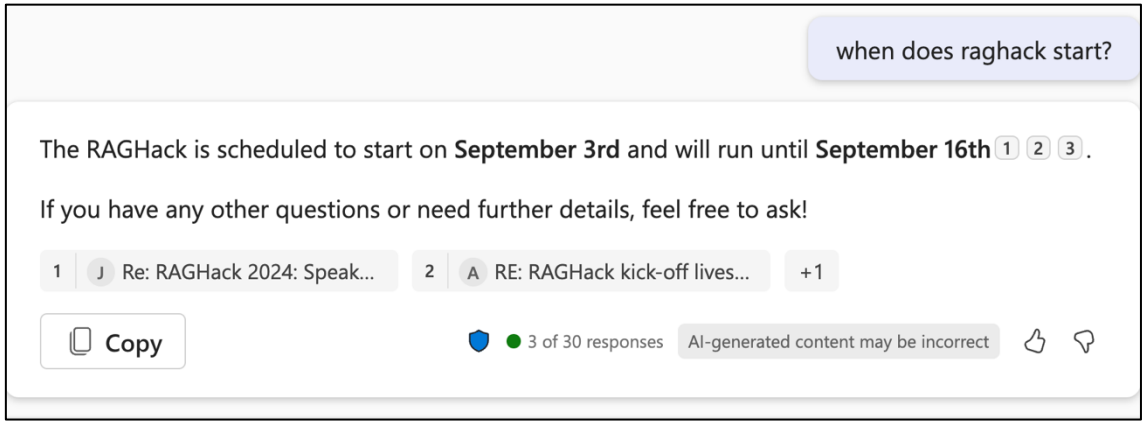
💵 High cost, time 🕰️

**Retrieval Augmented Generation**

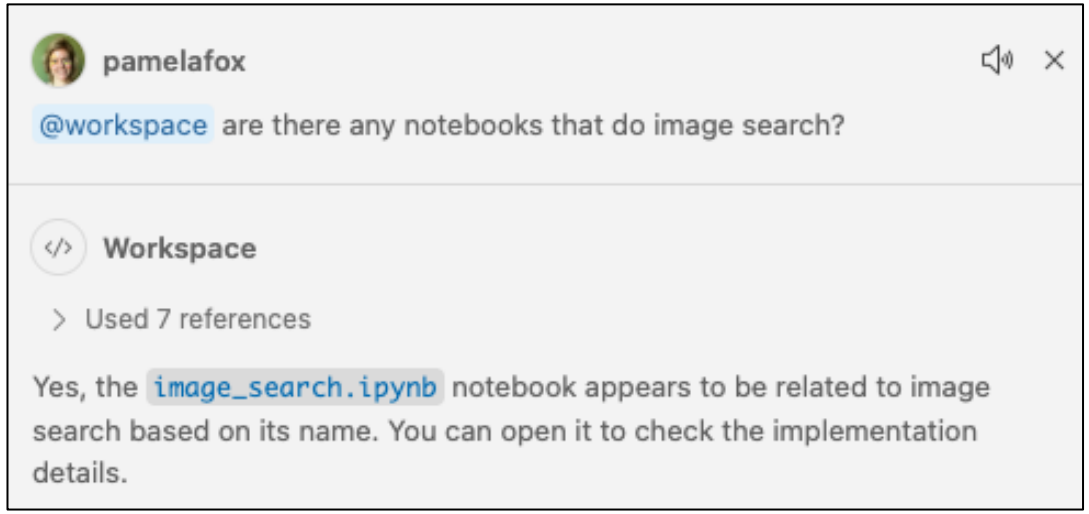
Learn new facts (temporarily)

# RAG in the wild

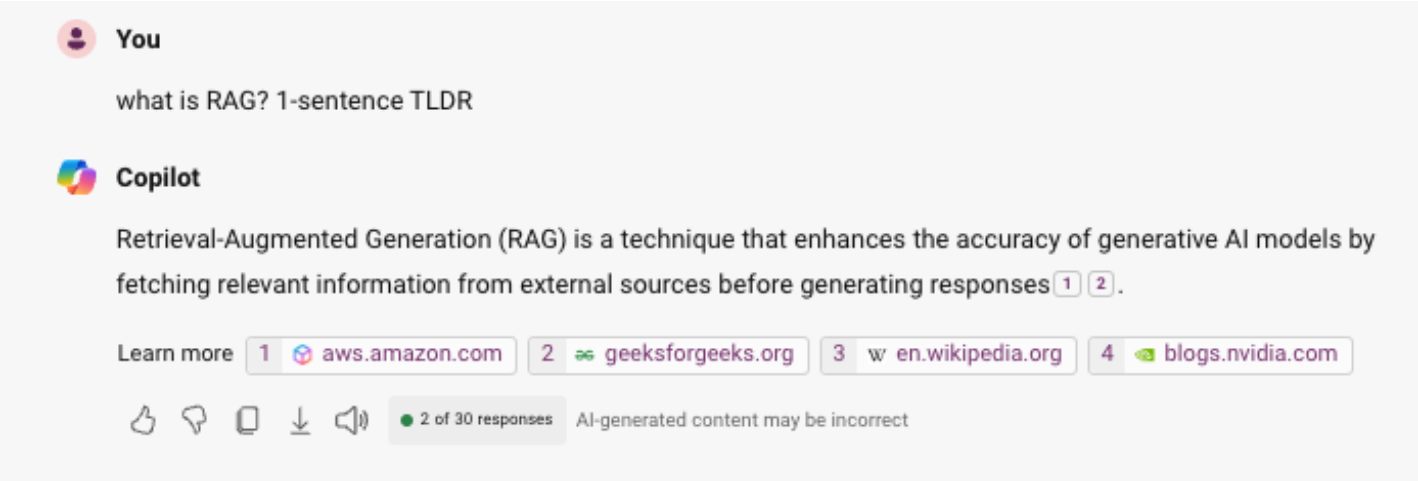
Teams Copilot (RAG on your chats)




GitHub Copilot (RAG on VS Code workspace)

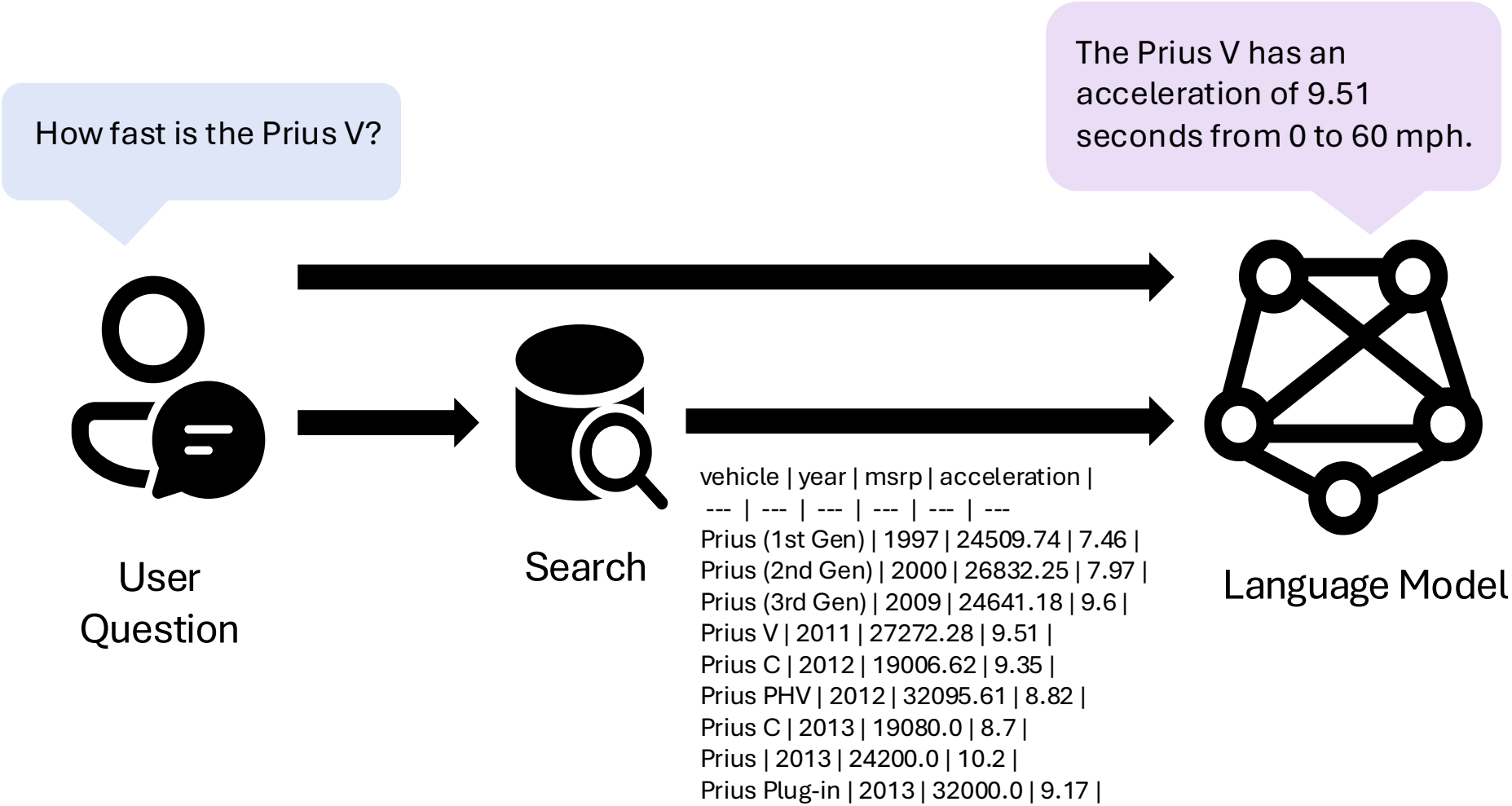



Bing + Copilot
(RAG on the web)



# RAG 101

# RAG: Retrieval Augmented Generation

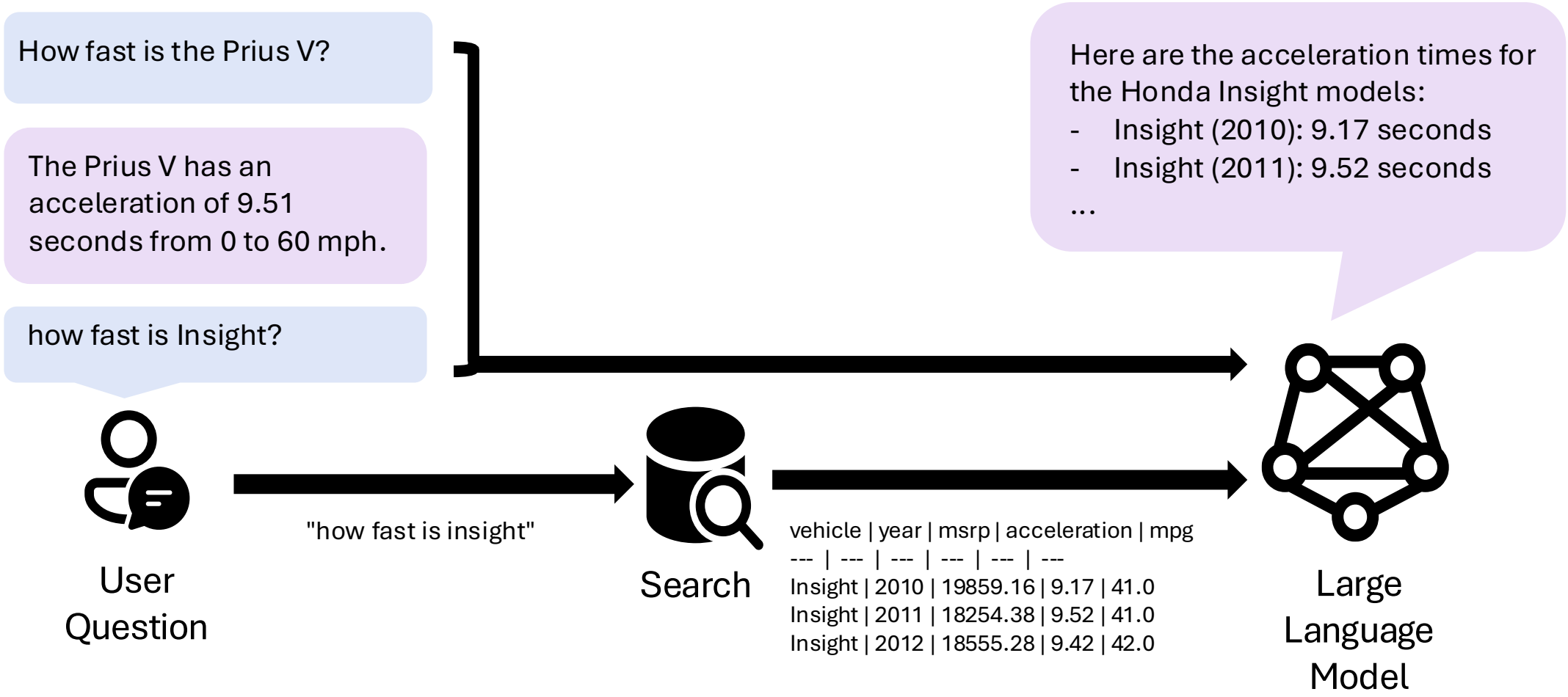


# RAG with OpenAI Python SDK

```
user_query = "How fast is the Prius V?"
retrieved_content = "vehicle | year | msrp | acceleration | mpg | class
 --- | --- | --- | --- | --- | ---
Prius (1st Gen) | 1997 | 24509.74 | 7.46 | 41.26 | Compact
Prius (2nd Gen) | 2000 | 26832.25 | 7.97 | 45.23 | Compact..."

response = openai.chat.completions.create(
  messages = [
   {
    "role": "system",
    "content": "You must answer questions according to sources provided."
   },
   {
    "role": "user",
    "content": user_query + "\n Sources: \n" + retrieved_content
   }
])
```

[rag_csv.py](rag_csv.py)

# RAG with multiturn support

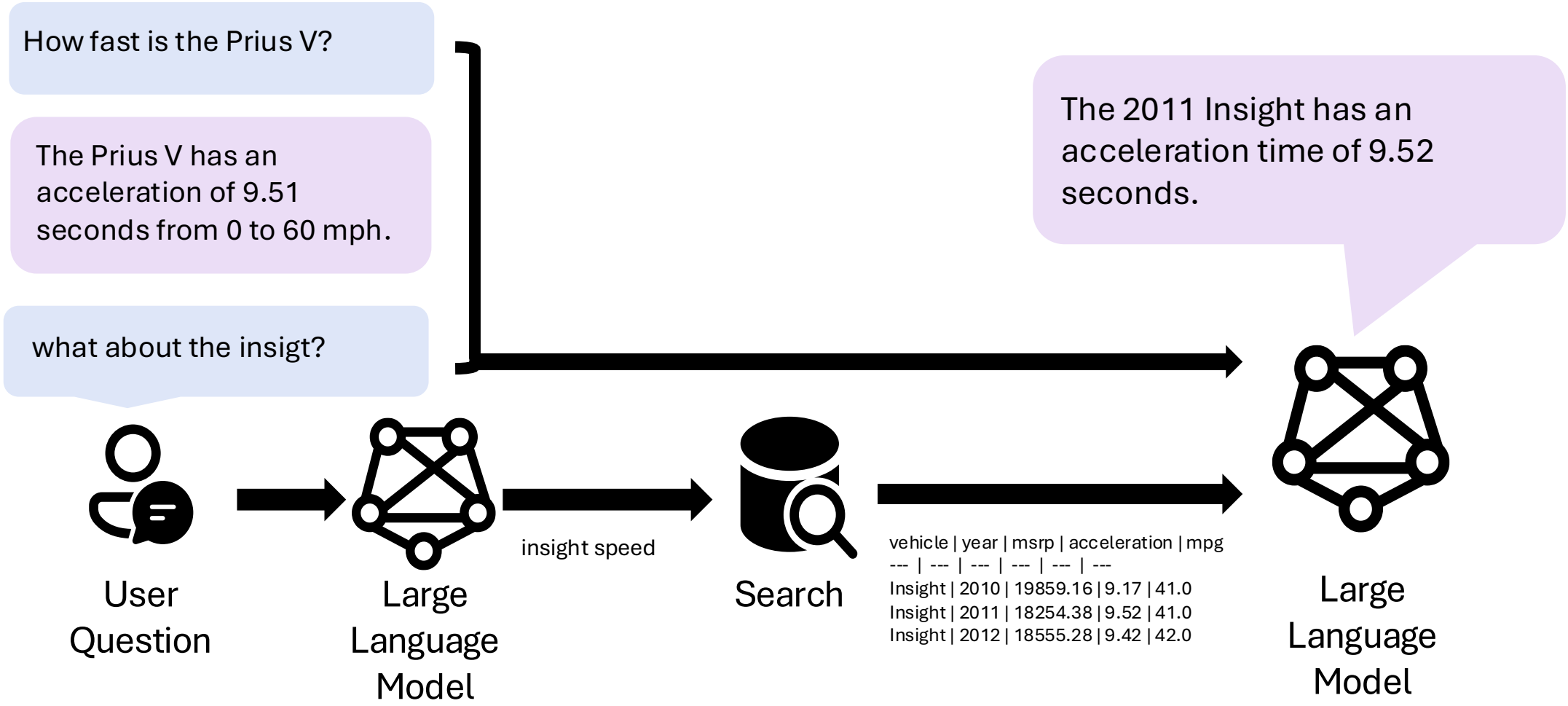


# RAG with multiturn support (Code)

```
messages = [{"role": "system", "content": SYSTEM_MESSAGE}]

while True:
  question = input("\nYour question: ")
  matches = search(question)

  messages.append({"role": "user", "content": f"{question}\nSources: {matches}"})
  response = client.chat.completions.create(
    model=MODEL_NAME,
    temperature=0.3,
    messages=messages
  )

  bot_response = response.choices[0].message.content
  messages.append({"role": "assistant", "content": bot_response})
```

rag_multiturn.py

# RAG with multiturn + query rewriting



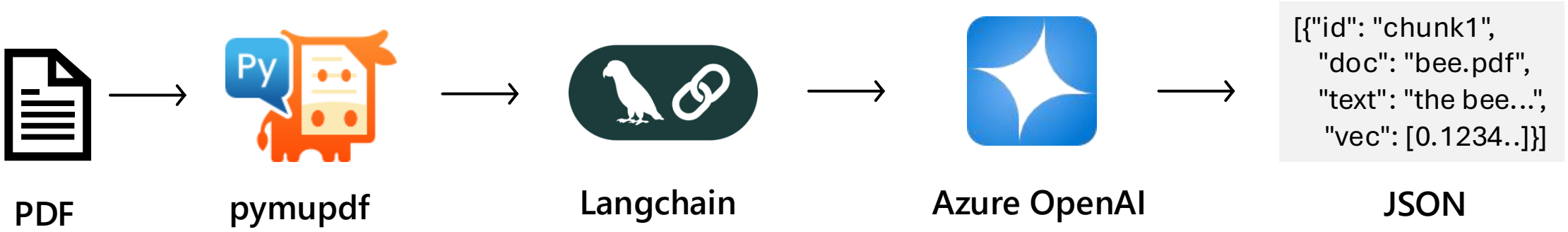
# RAG with multiturn + query rewriting (Code)

```
messages = [{"role": "system", "content": SYSTEM_MESSAGE}]

while True:
  question = input("\nYour question: ")
  response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
      {"role": "system", "content": QUERY_REWRITE_SYSTEM_MESSAGE},
      {"role": "user", "content": f"New question: {question}\nHistory:{messages}"}])
  search_query = response.choices[0].message.content

  matches = search(search_query)
  messages.append({"role": "user", "content": f"{question}\nSources: {matches}"})
  response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages)
  bot_response = response.choices[0].message.content
  messages.append({"role": "assistant", "content": bot_response})
```

rag_queryrewrite.py

# RAG document ingestion

For long/unstructured documents, we need an ingestion flow such as this one:




### Extract text from PDF

Other options for this step: Azure Document Intelligence, Langchain document loaders, OCR services, Unstructured, etc.

### Split data into chunks

Split text based on sentence boundaries and token lengths.

You could also use "semantic" splitters and your own custom splitters.

### Vectorize chunks

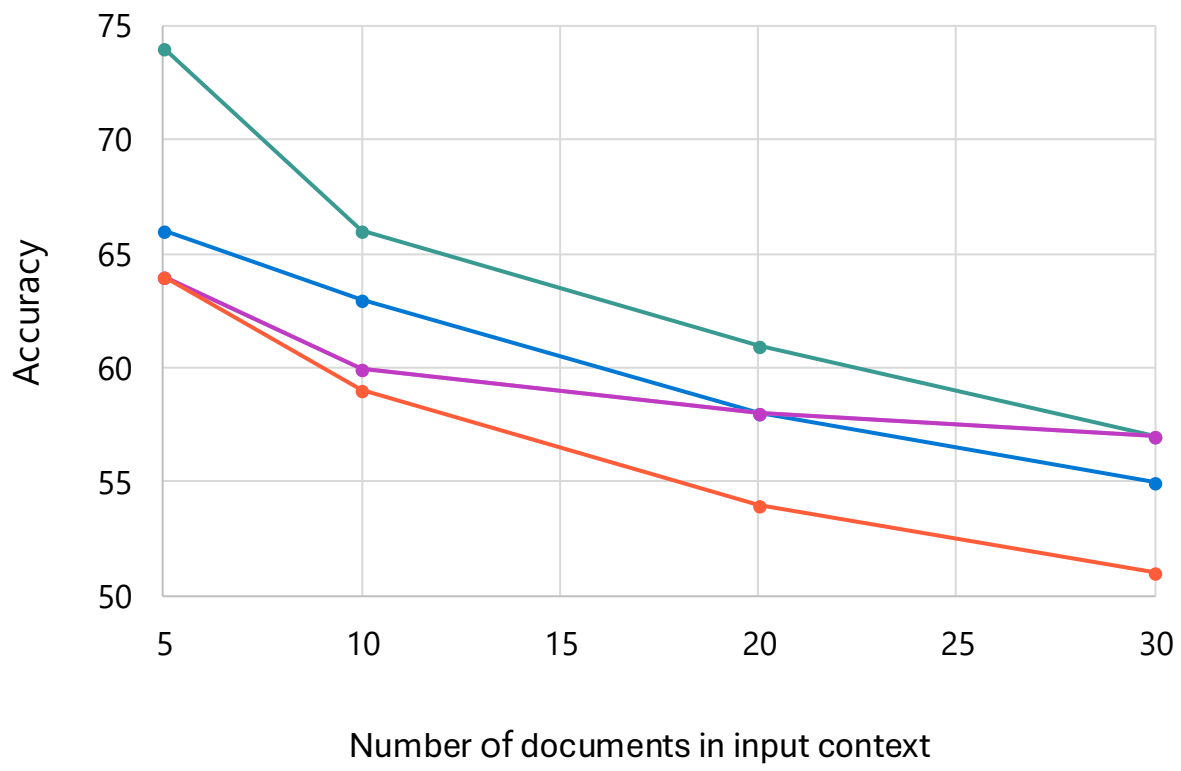
Compute embeddings using embedding model of your choosing.

### Store chunks

This is where you'd typically use a search service like Azure AI Search or a database like PostgreSQL.

# Why do we need to split documents?

1. LLMs have limited context windows (4K – 128K)
2. When an LLM receives too much information, it can get easily distracted by irrelevant details.
3. The more tokens you send, the higher the cost, the slower the response.




Source: Lost in the Middle: How Language Models Use Long Contexts, Liu et al. arXiv:2307.03172

# Optimal size of document chunk

## How big should chunks be?

| # of tokens per chunk | Recall@50 |
|---|---|
| 512 | **42.4** |
| 1024 | 37.5 |
| 4096 | 36.4 |
| 8191 | 34.9 |

Source: aka.ms/ragrelevance

A **token** is the unit of measurement for an LLM's input/output. ~1 token/word for English, higher ratios for other languages.
More on token ratios: aka.ms/genai-cjk

## Where to split chunks?

| Chunk boundary strategy | Recall@50 |
|---|---|
| Break at token boundary | 40.9 |
| Preserve sentence boundaries | 42.4 |
| 10% overlapping chunks | 43.1 |
| 25% overlapping chunks | **43.9** |

Source: aka.ms/ragrelevance

A chunking algorithm should also consider tables, and avoid splitting tables when possible.

# RAG document ingestion (Code)

```
filenames = ["data/California_carpenter_bee.pdf", "data/Centris_pallida.pdf"] all_chunks = []
for filename in filenames:
  md_text = pymupdf4llm.to_markdown(filename)

  text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
      model_name="gpt-4o", chunk_size=500, chunk_overlap=0)
  texts = text_splitter.create_documents([md_text])
  file_chunks = [{"id": f"{filename}-{(i + 1)}", "text": text.page_content}
      for i, text in enumerate(texts)]

  for file_chunk in file_chunks:
    file_chunk["embedding"] = (client.embeddings.create(
        model="text-embedding-3-small", input=file_chunk["text"])
        .data[0].embedding)

all_chunks.extend(file_chunks)
```

rag_documents_ingestion.py

# Simple RAG flow on documents (Code)

```
user_question = "where do digger bees live?"

docs = index.search(user_question)
context = "\n".join([f"{doc['id']}: {doc['text']}" for doc in docs[0:5]])

SYSTEM_MESSAGE = """
You must use the data set to answer the questions,
you should not provide any info that is not in the provided sources.
Cite the sources you used to answer the question inside square brackets.
The sources are in the format: <id>: <text>.
"""
response = client.chat.completions.create(
  model= "gpt-4o",
  temperature=0.3,
  messages=[
    {"role": "system", "content": SYSTEM_MESSAGE},
    {"role": "user", "content": f"{user_question}\nSources: {context}"}])
```

rag_documents_flow.py

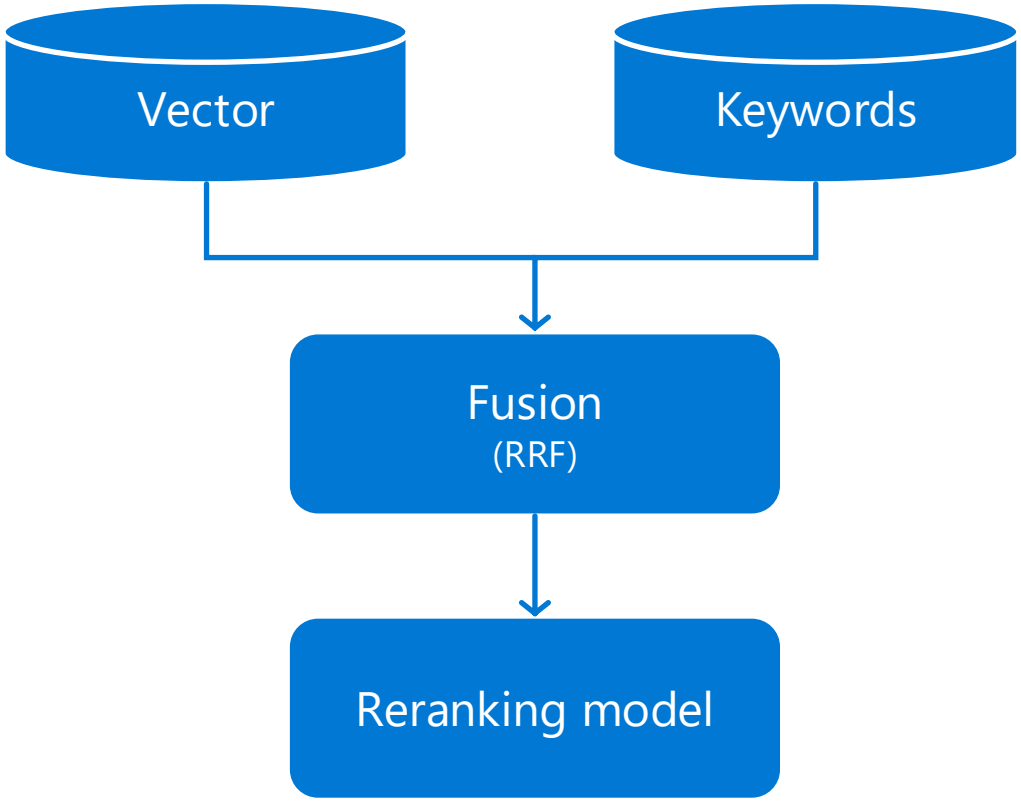
# RAG with hybrid retrieval

Complete search stacks do better:

- Hybrid retrieval (keywords + vectors) > vectors-only or keywords-only
- Hybrid + Reranking > Hybrid

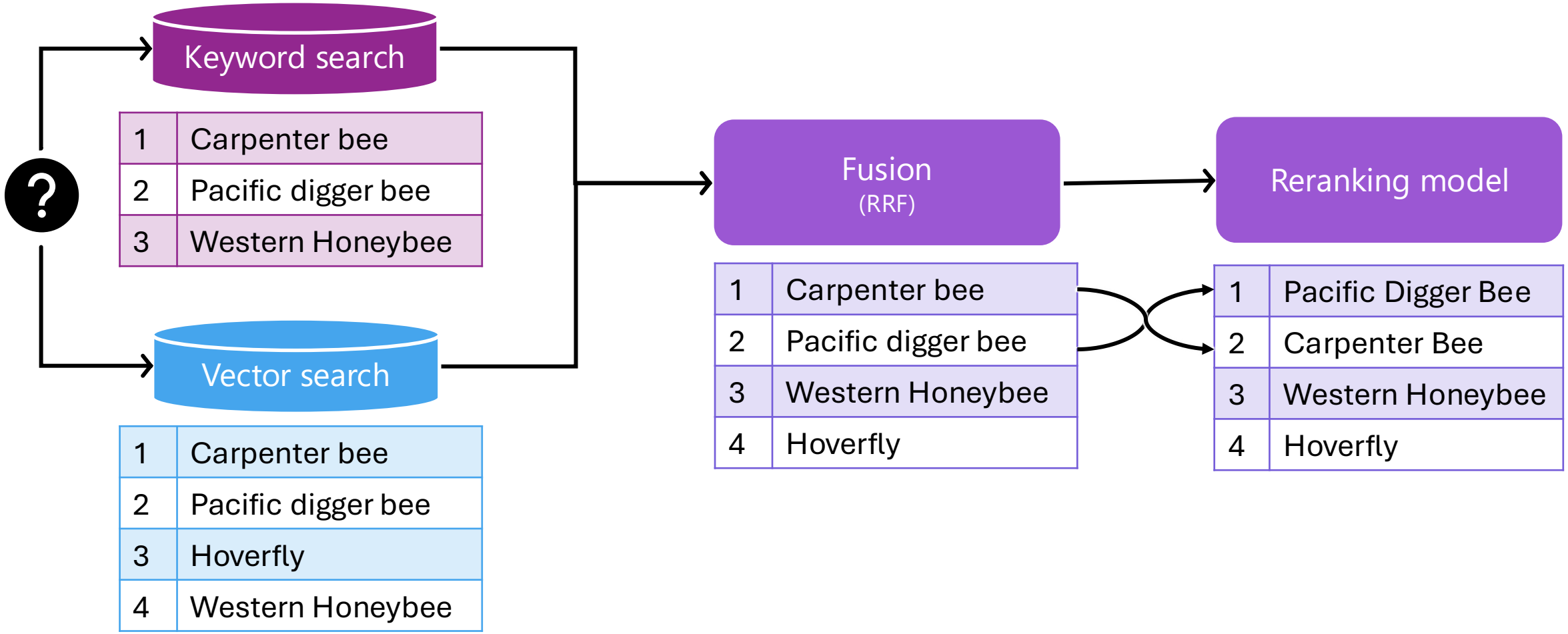
| search mode | groundedness | relevance |
|---|---|---|
| vector only | 2.79 | 1.81 |
| text only | 4.87 | 4.74 |
| hybrid | 3.26 | 2.15 |
| hybrid with reranking | 4.89 | 4.78 |

Source: [aka.ms/vector-search-not-enough](aka.ms/vector-search-not-enough)



# Hybrid retrieval flow

Question: "cute gray fuzzy bee"



# RAG with hybrid retrieval (Code)

```
def full_text_search(query, limit):

def vector_search(query, limit):

def reciprocal_rank_fusion(text_results, vector_results, alpha=0.5):

def rerank(query, retrieved_documents):
  encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
  scores = encoder.predict([(query, doc["text"]) for doc in retrieved_documents])
  return [v for _, v in sorted(zip(scores, retrieved_documents), reverse=True)]

def hybrid_search(query, limit):
  text_results = full_text_search(query, limit * 2)
  vector_results = vector_search(query, limit * 2)
  combined_results = reciprocal_rank_fusion(text_results, vector_results)
  combined_results = rerank(query, combined_results)
  return combined_results[:limit]
```

rag_documents_hybrid.py

# RAG data source types

## Database rows (Structured data)

You need a way to **vectorize** target columns with an **embedding model**.

You need a way to **search** the vectorized rows.

## Documents (Unstructured data)

PDFs, docx, pptx, md, html, images

You need an ingestion process for **extracting**, **splitting**, **vectorizing**, and **storing** document chunks.

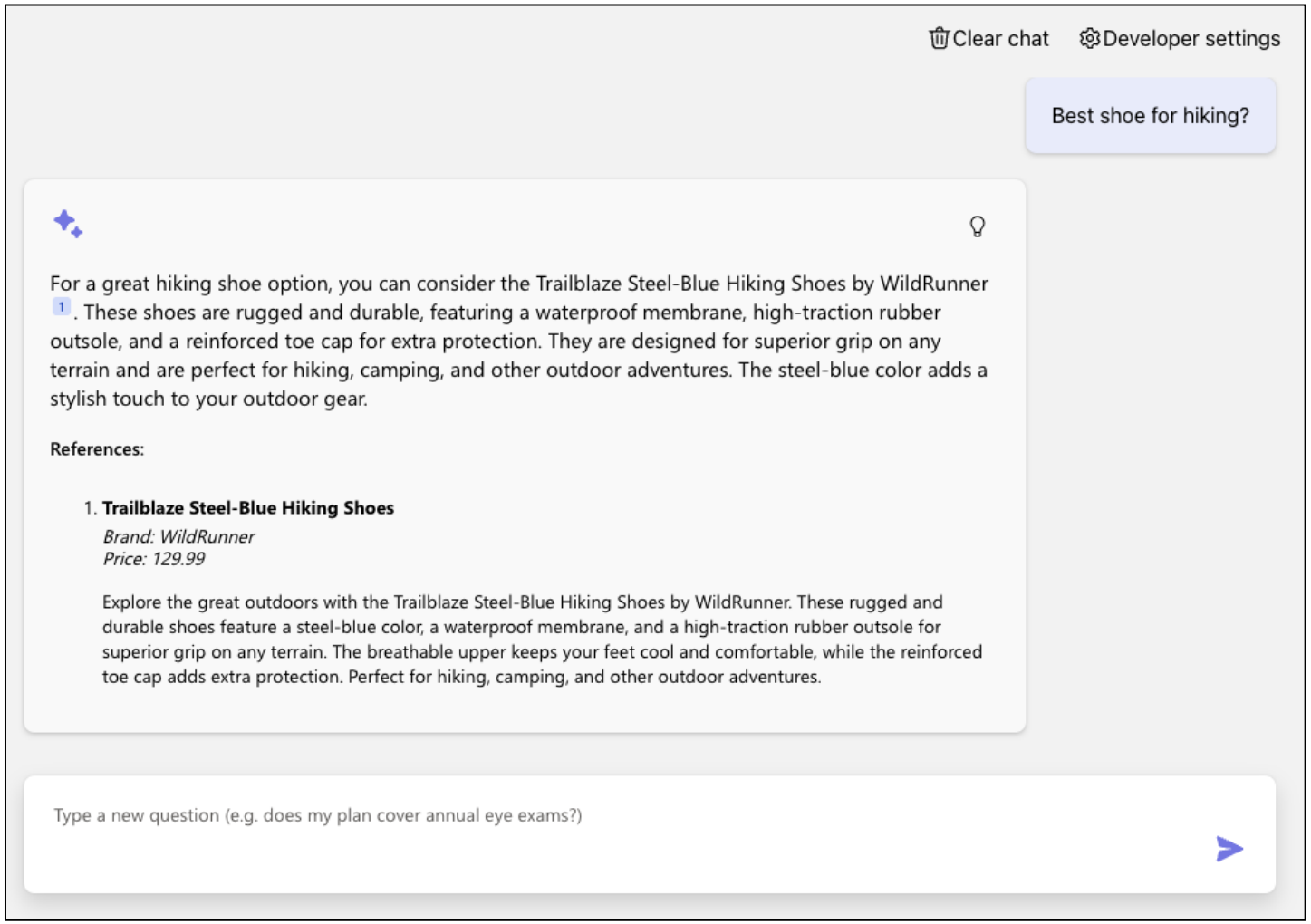
You need a way to **search** the vectorized **chunks**.

# RAG on PostgreSQL

# RAG on PostgreSQL in Python: Simplified

```
question = "any cheap climbing shoes?"

cur.execute("SELECT ... ")
results = cur.fetchall()
for result in results:
  formatted_results += f"## {result[1]}\n\n{result[2]}\n"

response = openai.chat.completions.create(
   messages = [
    {
     "role": "system",
     "content": "Answer questions according to sources provided."
    },
    {
     "role": "user",
     "content": question + "\n Sources: \n" + formatted _content
    }])
```

# RAG on PostgreSQL: Open-source template

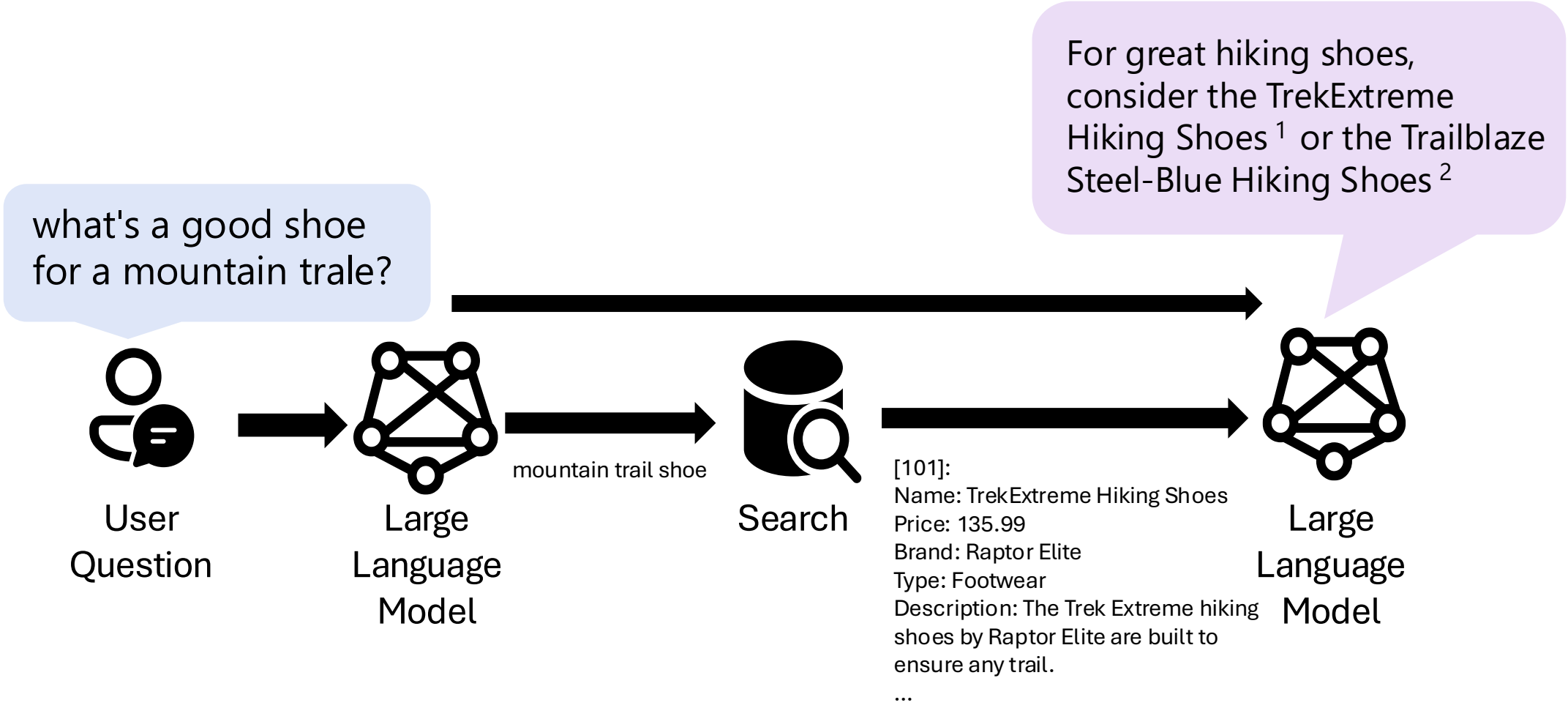



Azure OpenAI +
Azure PostgreSQL Flexible Server +
Azure Container Apps

Code:
aka.ms/rag-postgres

Demo:
aka.ms/rag-postgres/demo

# RAG on PostgreSQL: flow with query rewriting



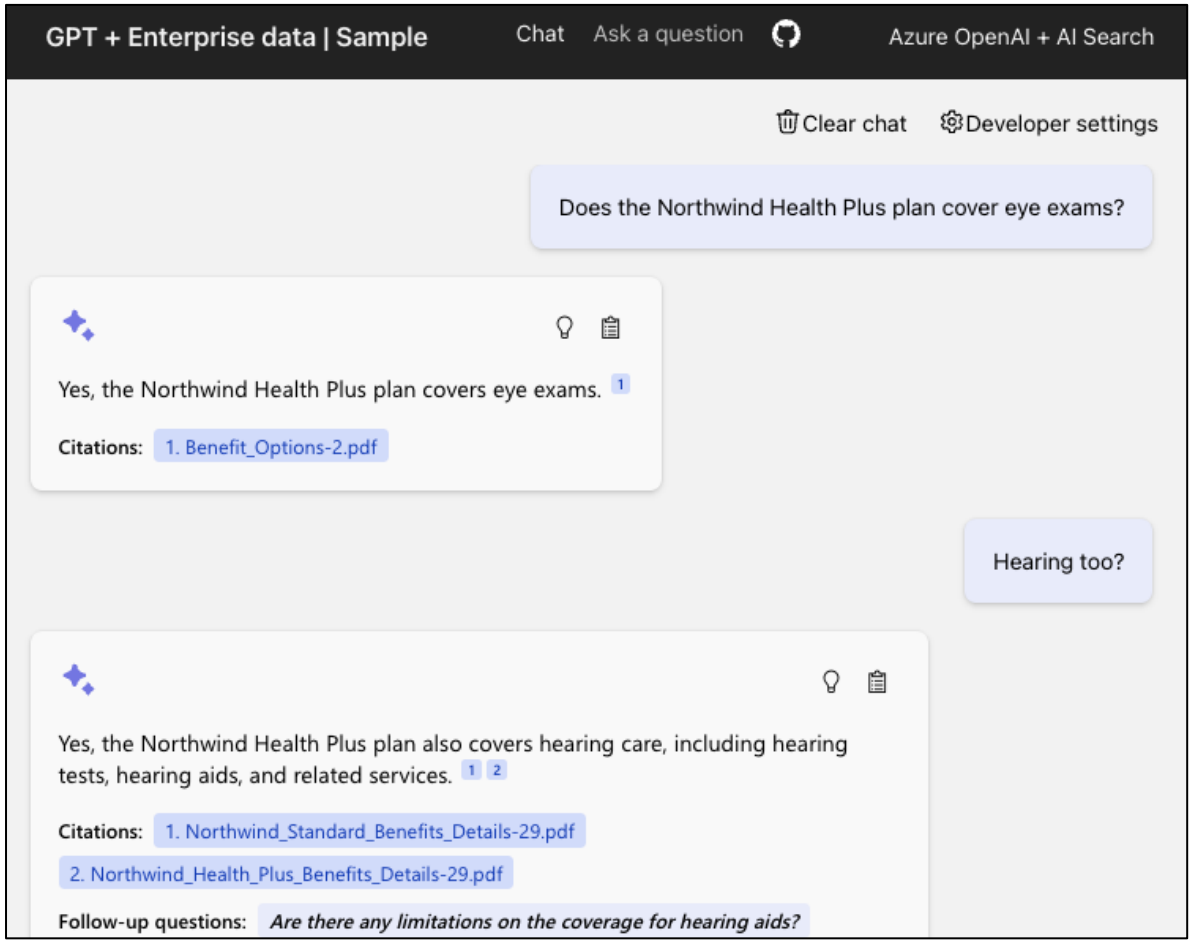
# RAG on Azure AI Search

# RAG with AI Search in Python: Simplified

```
user_question = "What does a product manager do?"

user_question_vector = get_embedding(user_question)
r = search_client.search(user_question,
    vector_queries=[VectorizedQuery(vector=user_question_vector, fields="vector")],
sources = "\n\n".join([f"[{doc['sourcepage']}]: {doc['content']}\n" for doc in r])
response = openai_client.chat.completions.create(
  model="gpt-4o",
  messages=[
     {"role": "system",
     "content": "Answer ONLY with the facts from sources below. Cite sources with brackets."""},
     {"role": "user",
      "content": user_question + "\nSources: " + sources}])
```

[rag-with-azure-ai-search-notebooks: rag.ipynb]

# RAG with AI Search: Open source template



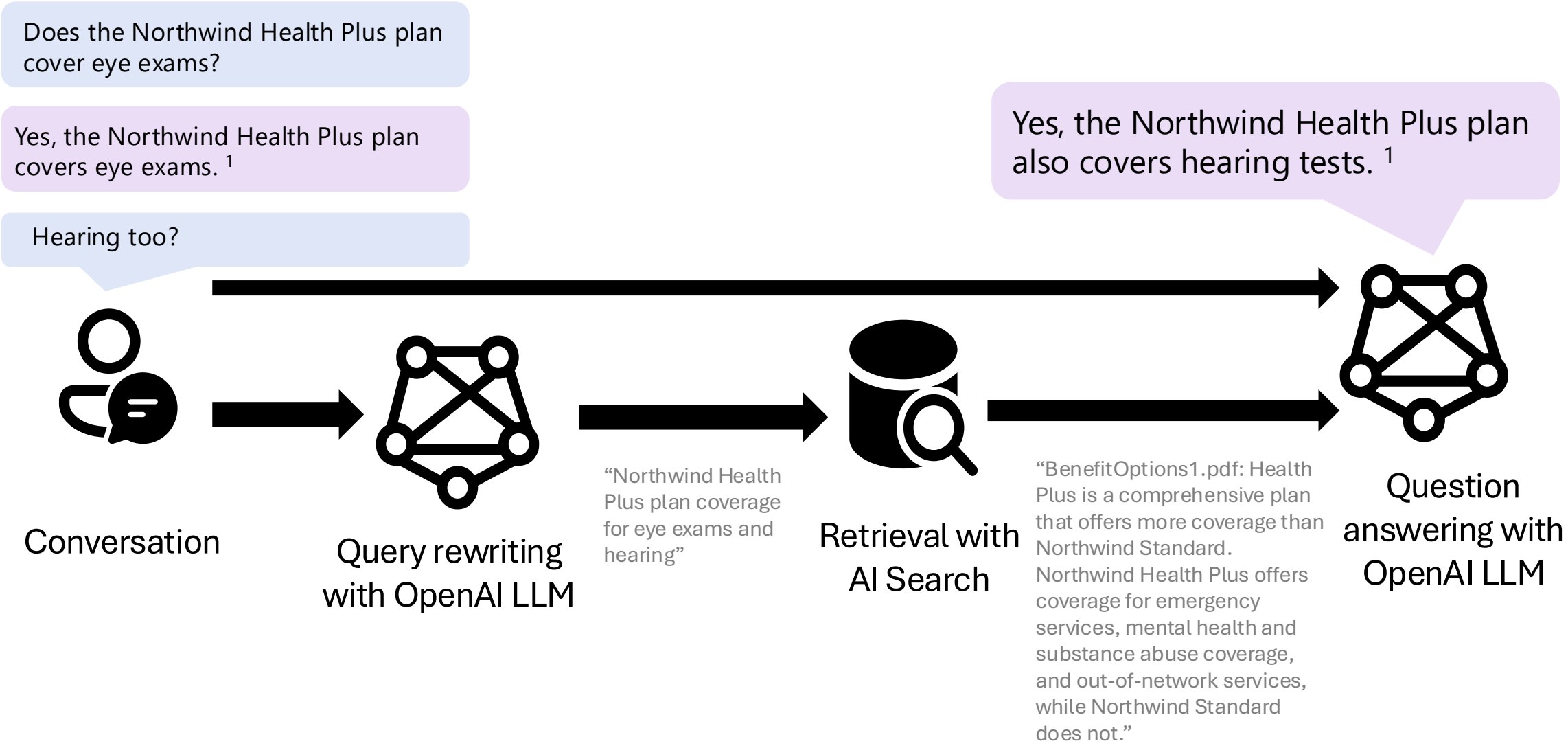

Azure OpenAI +
Azure AI Search +
Azure App Service/Container Apps

Supports simple and advanced flows
(Ask tab vs. Chat tab)

Code:
[aka.ms/ragchat](aka.ms/ragchat)

Demo:
[aka.ms/ragchat/demo](aka.ms/ragchat/demo)

# RAG with AI Search: flow with query rewriting



# RAG with AI Search: Data ingestion

The ingestion process is handled by a Python script:

Azure Blob Storage → Azure Document Intelligence → Python → Azure OpenAI → Azure AI Search

**Upload documents**

An online version of each document is necessary for clickable citations.

**Extract data from documents**

Supports PDF, HTML, docx, pptx, xlsx, images, plus can OCR when needed.

Local parsers also available for PDF, HTML, JSON, txt.

**Split data into chunks**

Split text based on sentence boundaries and token lengths.

Langchain splitters could also be used here.

**Vectorize chunks**

Compute embeddings using OpenAI embedding model of your choosing.

**Indexing**

- Document index
- Chunk index
- Both

# Learn more

# More ways to build a RAG app

| Component | Examples |
| --- | --- |
| **Ingestion**: Tools for processing data into a format that can be indexed and processed by LLM | Azure: Document Intelligence<br>Local: PyMuPDF, BeautifulSoup |
| **Retriever**: A knowledge base that can efficiently retrieve sources that match a user query<br>(Ideally supports both vector and full-text search) | Azure: Azure AI Search, Azure CosmosDB,<br>Local: PostgreSQL, Qdrant, Pinecone |
| **LLM**: A model that can answer questions based on the query based on the provided sources, and can include citations | OpenAI: GPT-4o, GPT-4.1-mini, GPT-5<br>Azure AI Studio: Meta Llama3, Mistral, Cohere R+<br>Anthropic: Claude 4.5<br>Google: Gemini 2.5 |
| **Orchestrator (optional)**: A way to organize calls to the retriever and LLM | Microsoft: Agent Framework<br>Community: Llamaindex, Langchain |
| **Features** | Chat history, Feedback buttons, Text-to-speech, User login, File upload, Access control, etc. |

# Watch more talks about RAG

**RAG Deep Dive (January 2025):** 11 streams about azure-search-openai-demo
aka.ms/ragdeepdive/watch

**RAGHack (August 2024):** 25+ streams about building RAG on Azure
aka.ms/raghack/streams

**RAG Time (March 2025):** Advanced topics on RAG with Azure AI Search
aka.ms/rag-time

**Building RAG from Scratch with GitHub Models**
aka.ms/rag-vs-code-github-models

# Next steps

Join upcoming streams →

Watch past recordings:

aka.ms/pythonai/resources

Come to office hours on Tuesdays in Discord:

aka.ms/pythonai/oh

🧠 Oct 7: LLMs

↖ Oct 8: Vector embeddings

🔍 Oct 9: RAG

🖼️ Oct 14: Vision models

🔢 Oct 15: Structured outputs

🛡️ Oct 16: AI Quality & Safety

🛠️ Oct 21: Tool calling

🤖 Oct 22: Agents

🔗 Oct 23: Model Context Protocol

Register @ aka.ms/PythonAI/series

# Thank you!